

v	p	e	Dijkstra (s)	Bellman-Ford (s)	Ratio
50	0.5	1238	0.00023	0.0037	16.27034
100	0.5	5021	0.00075	0.02855	38.15194
150	0.5	11296	0.00168	0.10284	61.27237
200	0.5	20178	0.00314	0.2446	77.77293
250	0.5	31240	0.00474	0.48587	102.53669
v	p	e	Dijkstra (s)	Bellman-Ford (s)	Ratio
100	0.01	208	0.00059	0.00194	3.28642
100	0.1	1076	0.00051	0.00727	14.3322
100	0.3	3153	0.00063	0.01919	30.26774
100	0.5	4945	0.00076	0.02947	38.78584
100	0.7	7023	0.00088	0.04169	47.39383
100	0.9	8963	0.00103	0.0532	51.66138
100	0.99	9813	0.00116	0.05894	50.8639

這次對兩個演算法的比較設計兩種方向：第一種是增加頂點數量，使邊的數量也隨之增加；第二種則固定頂點數量，透過提高邊的生成機率來增加邊數，每個 graph 固定產生一次，之後使用相同的 graph 分別執行 Dijkstra 與 Bellman-Ford 10 次，最後取平均執行時間。。

在第一組實驗中，隨著 V 增加，兩種演算法的執行時間皆增加，但 Bellman-Ford 的增加幅度明顯較大。當 $V=50, E=1238$ 時，Bellman-Ford 的執行時間約為 Dijkstra 的 16.27 倍；當 $V=250, E=31240$ 時，這個差距增加至 102.54 倍。顯示隨著圖的規模增加，Bellman-Ford 的執行時間相較於 Dijkstra 增長得更快。

第一組實驗中固定 $p=0.5$ 。由於圖為有向圖，隨機邊的期望數量約為

$$E \approx pV(V-1) = 0.5V(V-1), E \approx 0.5V(V-1),$$

另外程式還會加入 $V-1$ 條保證存在的邊，但這不影響其 $O(V^2)$ 的成長量級。因此 Dijkstra 的時間複雜度

$$O(V^2 + E)$$

仍可視為 $O(V^2)$ ；Bellman-Ford 的時間複雜度為

$$O(VE), O(VE),$$

代入 $E=O(V^2)$ 後可得 $O(V^3)$ 。因此，實驗中 Bellman-Ford 隨著 V 增加而比 Dijkstra 增長更快的結果，與兩者的理論時間複雜度一致。

第二組實驗則固定 $V=100$ ，透過改變 p 來增加邊數。此時

$$E \approx pV(V-1), E \approx pV(V-1),$$

因此隨著 p 增加, EE 也會增加。Dijkstra 的複雜度為 $O(V^2+E)$, 而 Bellman-Ford 為 $O(VE)$ 。在 V 固定的情況下, 增加一條邊對 Dijkstra 只增加與 EE 有關的一個項目; 對 Bellman-Ford 則會使 VE 增加 V 。因此 Bellman-Ford 對邊數增加更加敏感, 這與實驗結果相符。

值得注意的是, 在 $p=0.01$ 到 $p=0.1$ 時, Dijkstra 的測量時間由 0.00059 秒略降至 0.00051 秒。由於差異非常小, 且後續隨著邊數增加又呈現整體上升趨勢, 因此此現象較可能來自執行環境或測量誤差, 而不代表增加邊數會使 Dijkstra 的實際時間複雜度下降。

從執行時間來看, Bellman-Ford 在本次測試中明顯慢於 Dijkstra。然而, 兩種演算法的適用條件並不完全相同。Dijkstra 使用 greedy strategy, 在所有邊權重皆為非負的情況下, 可以將目前距離最小的頂點視為已確定; 但當圖中存在負權邊時, 之後可能透過負邊找到更短的路徑, 因此原本已確定的距離可能不再正確。Bellman-Ford 則透過反覆 relaxation 所有邊, 因此能夠處理包含負權邊的圖。此外, Bellman-Ford 還可以透過額外的一輪 relaxation 偵測 reachable negative cycle。

例如考慮以下圖形:

```
graph = {
  'A': [('B', 1), ('C', 2)],
  'B': [('D', 1)],
  'C': [('B', -3)],
  'D': []
}
```

從 A 出發時, Dijkstra 首先得到 B 的距離為 1、C 的距離為 2, 因此會先處理 B。此時 B 被視為已確定, 並得到 D 的距離為 2。然而之後處理 C 時, 可以透過負權邊 $C \rightarrow B$ 將 B 的距離更新為

$$2 + (-3) = -1.$$

此時 B 雖然已經變成 -1, 但 Dijkstra 不會再次處理 B, 因此 D 的距離仍為 2。另一方面, Bellman-Ford 可以透過多次 relaxation 找到

$A \rightarrow C \rightarrow B \rightarrow D$

的距離

$$2 - 3 + 1 = 0,$$

因此得到正確的結果。

這個實驗顯示, Bellman-Ford 雖然在本次測試中的執行時間較長, 但它能處理 Dijkstra 無法保證正確處理的負權邊, 並且能夠偵測 reachable negative cycle。因此兩種演算法除了執行效率之外, 在適用條件上也存在重要差異。